Secure Authentication System

Secure Login & Password Management

Technology Stack: Node.js, Express.js, MongoDB

1. Introduction

1.1 Overview

backend web application built using Node.js and Express.js, with MongoDB as its primary database. implement an authentication system that prioritises user security at every stage of interaction. The system handles user registration, email verification, secure login, JWT-based session management, and a complete password reset workflow.

1.2 Objectives

- Implement secure user registration with bcryptjs password hashing.
- Enable email-based verification using a one-time code delivered via Nodemailer.
- Authenticate users via hashed password comparison and issue signed JSON Web Tokens (JWT).
- Provide a time-limited, token-based password reset flow secured with SHA-256 hashing via Node.js crypto.
- Ensure that no sensitive data is exposed in the database, API responses, or email communications.

1.3 Technology Stack

Component	Role in System
Node.js	JavaScript runtime environment powering the server-side application.
Express.js	Minimal web framework used to define RESTful API routes and middleware.
MongoDB	NoSQL document database for persistent storage of user records.
bcryptjs	Library for hashing passwords using the adaptive bcrypt algorithm.
JSON Web Token (JWT)	Stateless token standard used for authenticating active sessions.
Node.js crypto	Built-in module for generating and hashing password reset tokens via SHA-256.
Nodemailer	Email delivery library used to send verification codes and reset links.
Postman	API testing tool used to simulate and validate all endpoints.

1.4 Scope

This report covers the complete backend authentication module. It does not address the frontend user interface. All API interactions are demonstrated using Postman, and all database states are verified using the MongoDB shell.

2. System Workflow

The authentication system is divided into four interconnected workflows: Registration, Email Verification, Login, and Password Reset. Each workflow is designed to be stateless, tamper-resistant, and minimal in the data it exposes.

2.1 User Registration Flow

When a user submits their registration credentials, the system performs the following sequence of operations:

1. **Step 1:** The client sends a POST request to /bidsphere/user/register with username, email, and password fields.
2. **Step 2:** The server validates that all fields are present. If any field is missing, a 400 error is returned immediately.
3. **Step 3:** A database lookup is performed to check whether the provided email already exists. Duplicate registrations are rejected.
4. **Step 4:** The plaintext password is passed to bcryptjs with a salt factor of 10, generating a one-way hashed value that is then stored in the database.
5. **Step 5:** A six-digit numeric verification code is generated and dispatched to the user's email via Nodemailer. This code is stored in the database in hash sha-256 form.
6. **Step 6:** The user record is written to MongoDB with isVerified set to false, indicating the account is pending email confirmation.

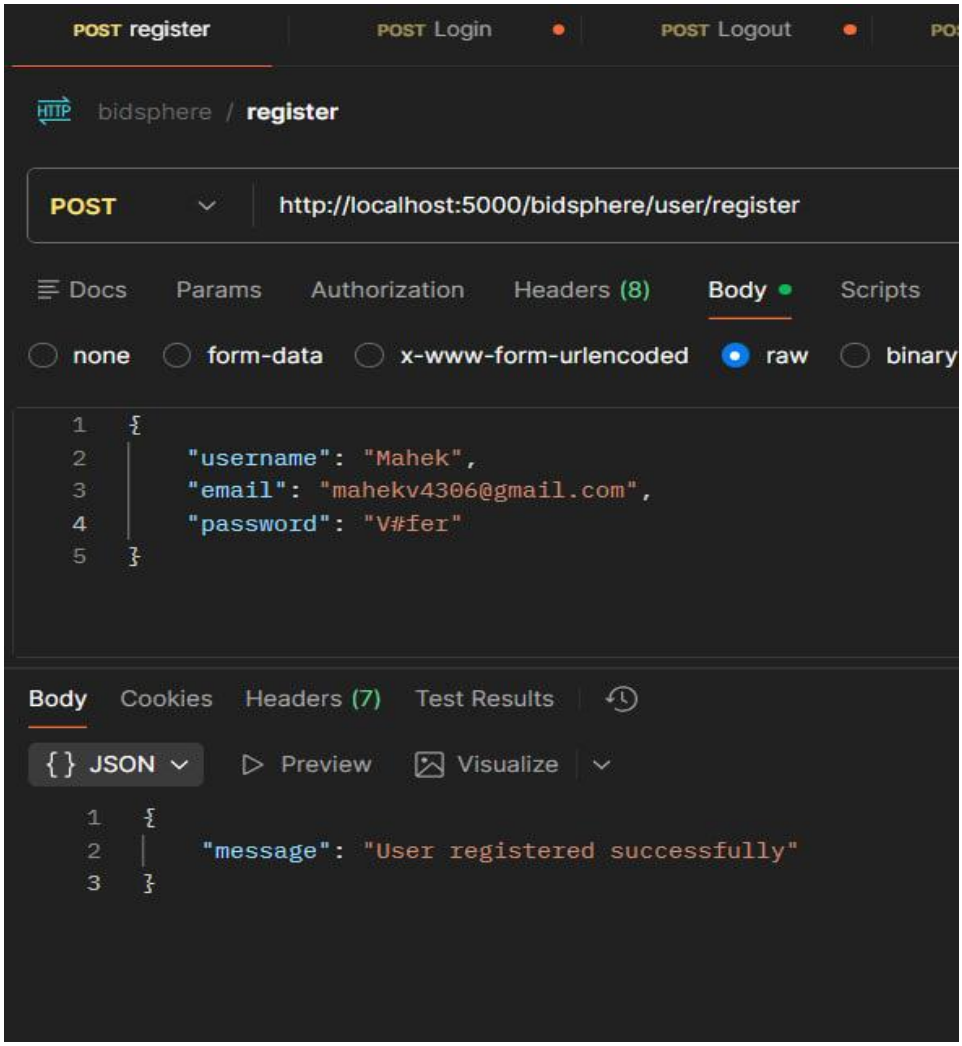

Figure 1: Postman — POST /register request and successful response

```
secure-login> db.users.find({})
[
  {
    _id: ObjectId('69e509ea943e14b34ffe0b25'),
    username: 'Mahek',
    email: 'mahekv4306@gmail.com',
    password: '$2b$10$YuJ3II9/dZnV7.efaGiOf.Hgt4y/i4WKYd5KcCvhyffyttN9ECfT2',
    verificationCode: 'eb9871dc8a48ad67bbd565820e2b1609250c79a962f3237c476d80253da5955b',
    verificationCodeExpires: ISODate('2026-04-19T17:09:22.819Z'),
    resetPasswordToken: null,
    resetPasswordExpires: null,
    isVerified: false,
    createdAt: ISODate('2026-04-19T16:59:22.835Z'),
    updatedAt: ISODate('2026-04-19T16:59:22.835Z'),
    __v: 0
  }
]
secure-login>
```

Figure 2: MongoDB — Newly created user document with bcrypt-hashed password and isVerified: false, verificationCode sent and stored hash form in databse

2.2 Email Verification Flow

Account activation is completed through a separate verification step. This two-step approach prevents unauthorised accounts and confirms the user's ownership of the email address.

7. **Step 1:** A formatted HTML email containing a six-digit OTP is delivered to the user's registered inbox via Nodemailer.
8. **Step 2:** The user submits the code along with their email to POST /bidsphere/user/verifyemail.
9. **Step 3:** The server retrieves the user record by email. If found, the submitted code is hashed using **crypto** and compared with the hashed verification code stored in the database; if the values match and the code has not expired, the isVerified flag is updated to true.
10. **Step 4:** A branded welcome email is dispatched confirming successful activation.

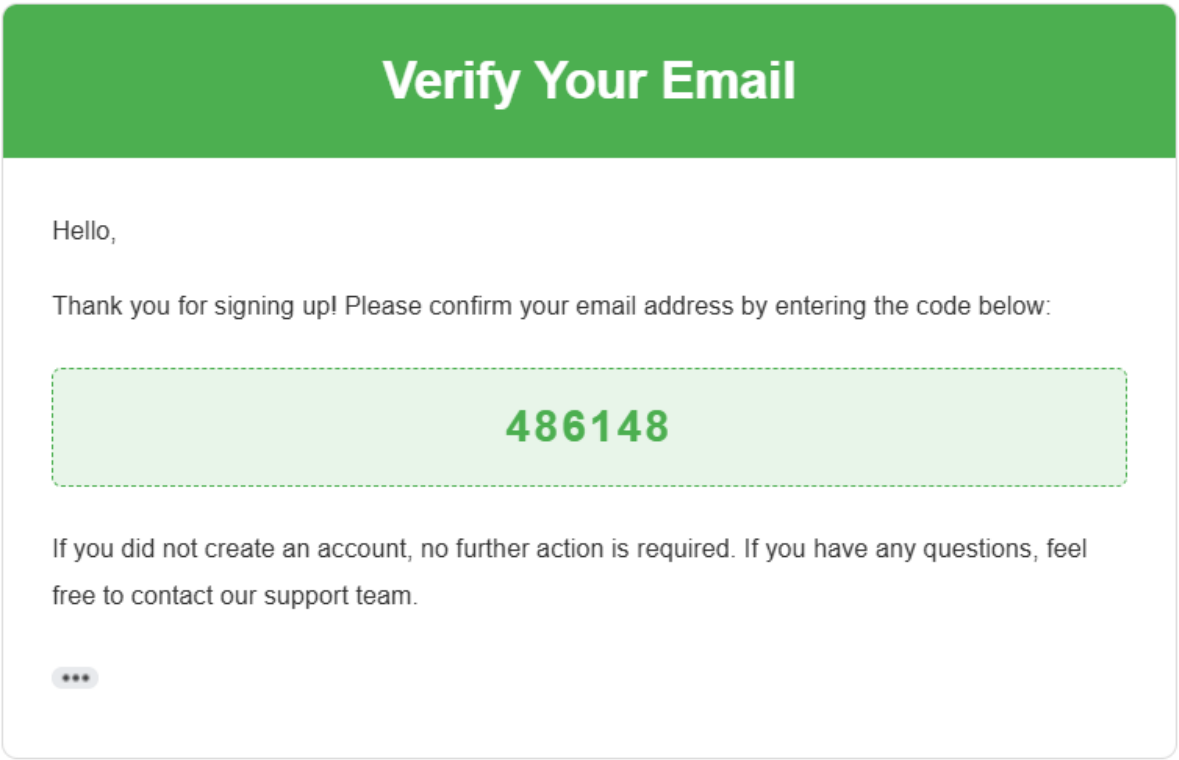

Figure 3: Gmail — Verification email with one-time code delivered to the user's inbox

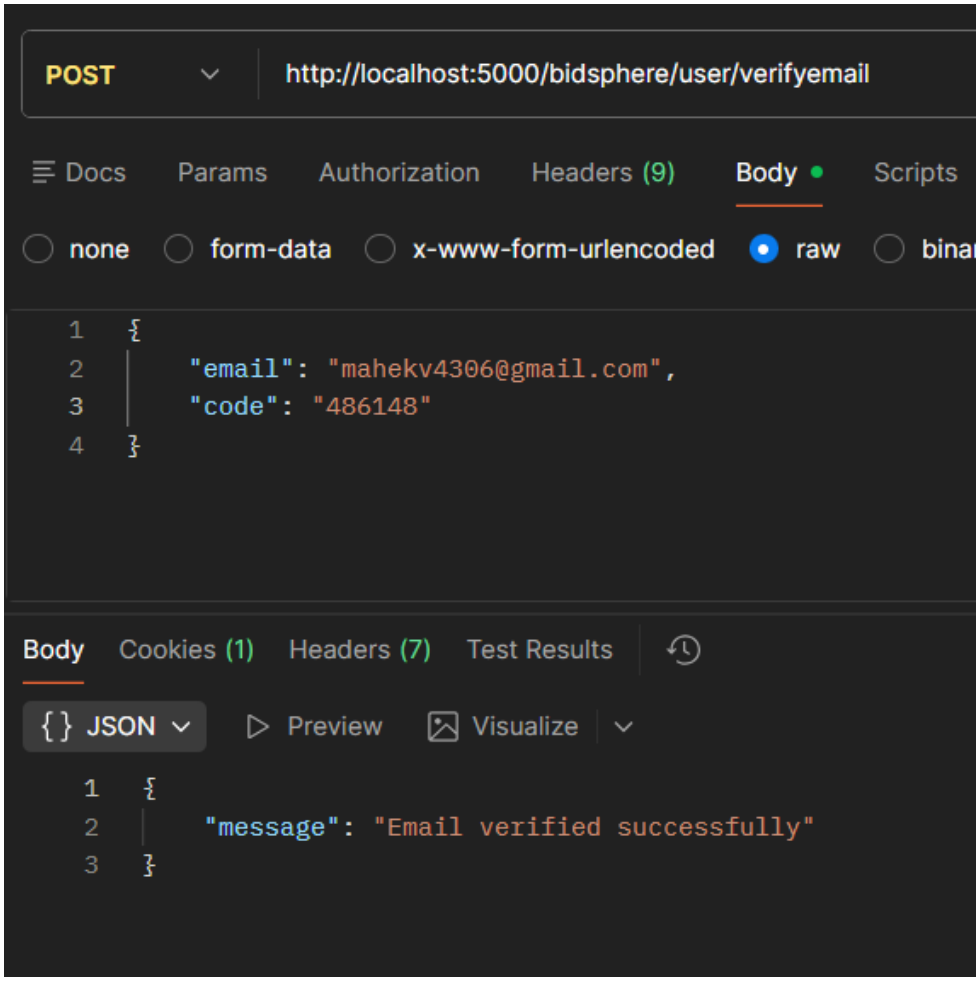

Figure 4: Postman — POST /verifyemail confirming email verification is successful

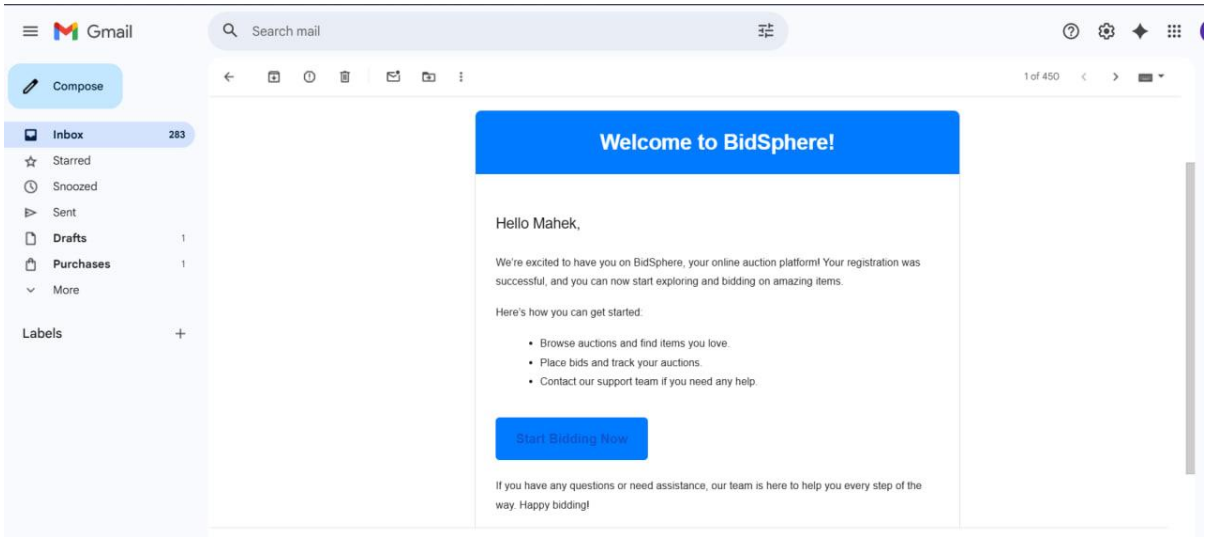

Figure 5: Gmail — Welcome email dispatched after successful account activation

```
secure-login> db.users.find({})
[
  {
    _id: ObjectId('69e509ea943e14b34ffe0b25'),
    username: 'Mahek',
    email: 'mahekv4306@gmail.com',
    password: '$2b$10$YuJ3II9/dZnV7.efaGiOf.Hgt4y/i4WKYd5KcCvhyffyttN9ECfT2',
    verificationCode: null,
    verificationCodeExpires: null,
    resetPasswordToken: null,
    resetPasswordExpires: null,
    isVerified: true,
    createdAt: ISODate('2026-04-19T16:59:22.835Z'),
    updatedAt: ISODate('2026-04-19T17:02:04.260Z'),
    __v: 0
  }
]
secure-login>
```

Figure 6: MongoDB — User document updated with isVerified: true after email confirmation. And verification code and it's expiry value null

2.3 Secure Login Flow

The login process implements a layered validation strategy that prevents information leakage while ensuring only fully verified, credential-matched users receive an authenticated session token.

11. **Step 1:** The client submits a POST request to /bidsphere/user/login with email and password.
12. **Step 2:** The server checks for an existing token cookie. If a session is already active, login is blocked to prevent token duplication.
13. **Step 3:** The user record is retrieved from MongoDB by email. A generic "Invalid email or password" message is returned for both non-existent users and mismatched passwords — deliberately avoiding enumeration of valid accounts.
14. **Step 4:** bcrypt.compare() is used to securely compare the plaintext password against the stored hash without ever decrypting the hash.
15. **Step 5:** If the isVerified flag is false, access is denied even if the password is correct.
16. **Step 6:** On successful authentication, a signed JWT is generated using jsonwebtoken with a 7-day expiry and set as an HttpOnly, Secure, SameSite=None cookie to prevent XSS and CSRF attacks.

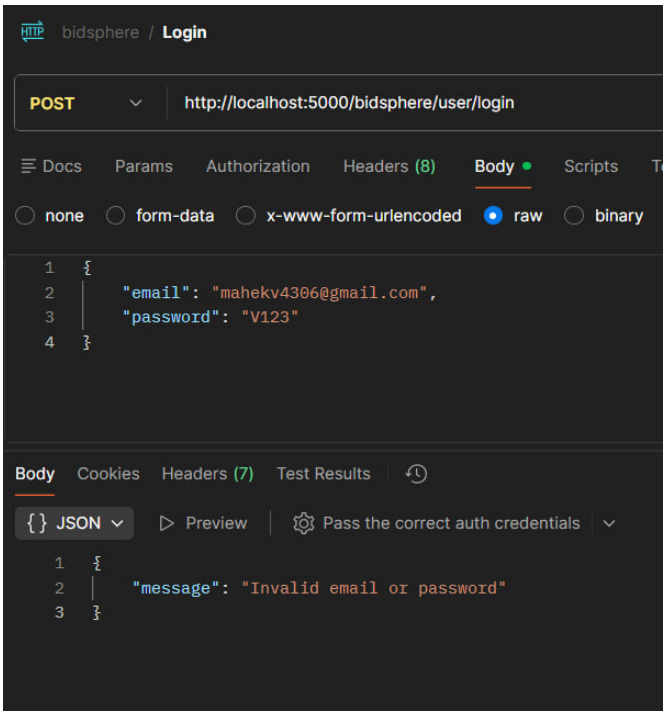

Figure 7:: Postman — Login attempt with incorrect password returning a generic error response

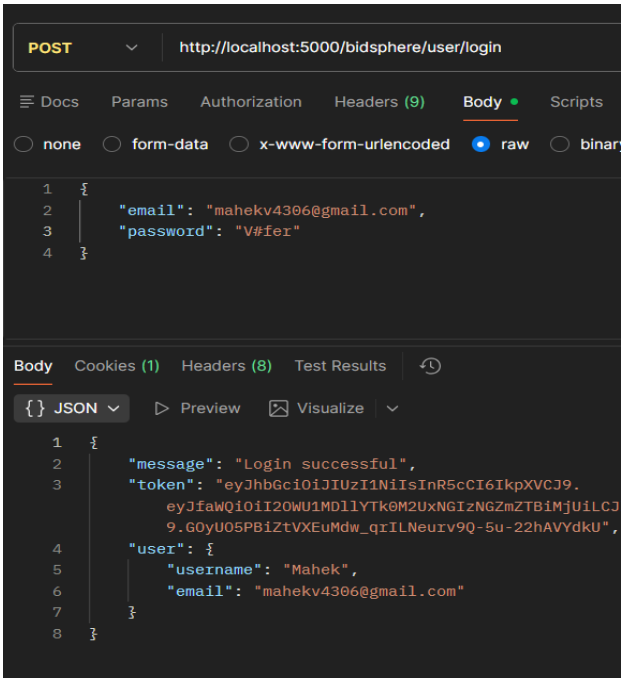

Figure 8: Postman — Successful login returning a signed JWT token with user details

2.4 Password Reset Flow

The password reset mechanism is designed to be both usable and secure. It uses a two-phase token system where the raw token is sent by email but only a SHA-256 hash of that token is stored in the database, mitigating the impact of any database breach.

Phase 1 — Requesting a Reset Link

17. **Step 1:** The user submits their email to POST /bidsphere/user/forgetpwd.
18. **Step 2:** The server uses crypto.randomBytes(32) to generate a cryptographically random 64-character hexadecimal token.
19. **Step 3:** This raw token is immediately hashed using SHA-256 via crypto.createHash('sha256'). Only the hash is stored in MongoDB. The raw token is never persisted.
20. **Step 4:** A 15-minute expiry timestamp is stored alongside the hash as resetPasswordExpires.
21. **Step 5:** A reset URL embedding the raw token and the user's email is sent to the inbox via Nodemailer.

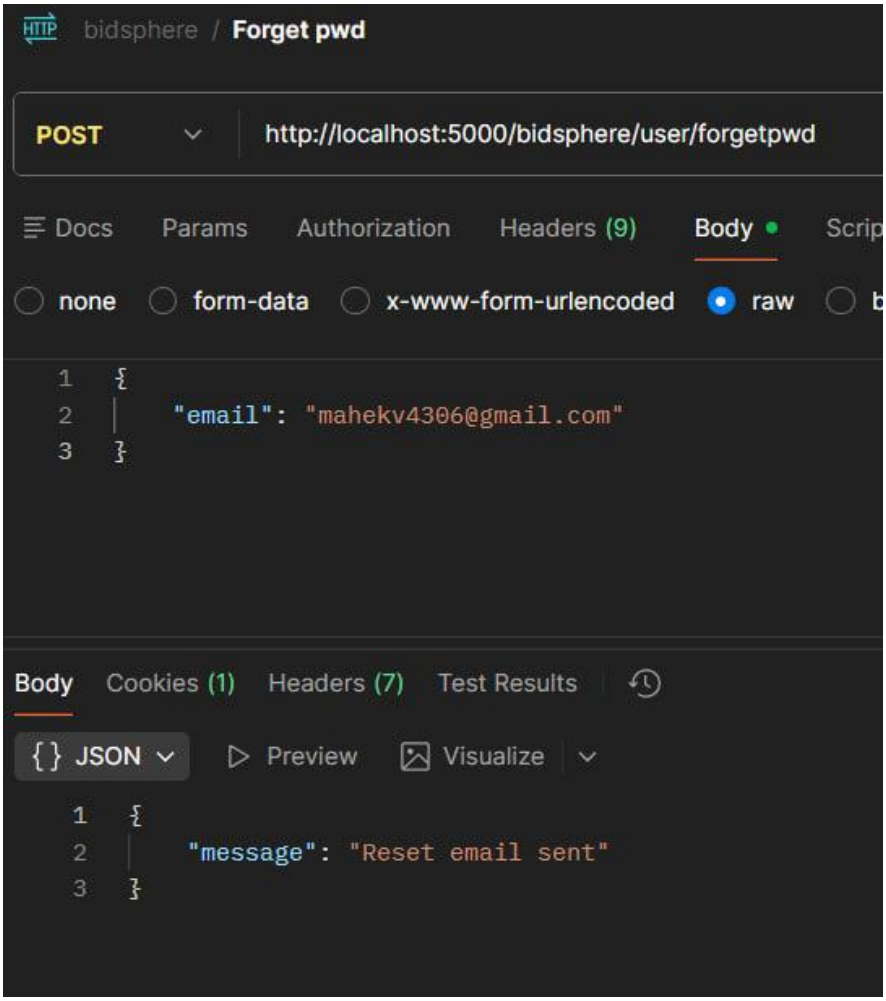

Figure 9: Postman — POST /forgetpwd request triggers a password reset email

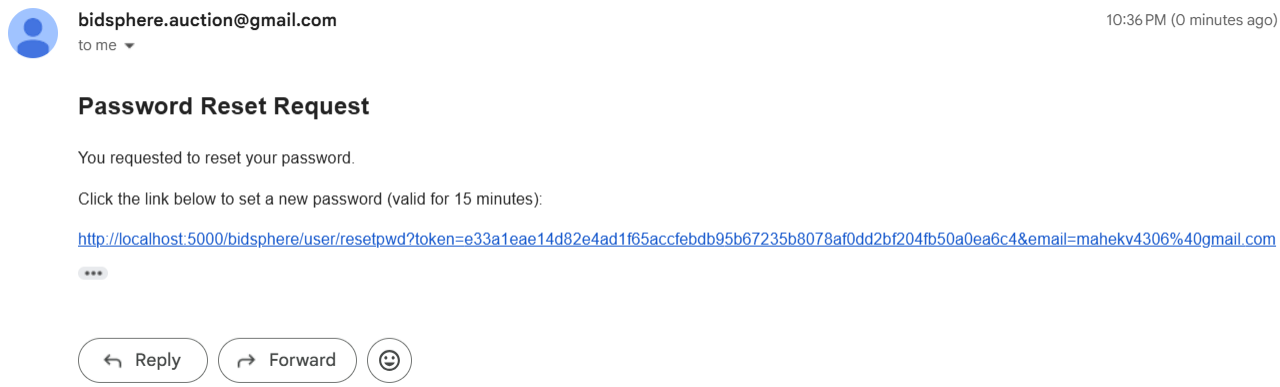

Figure 10: Gmail — Password reset email with tokenised link valid for 15 minutes

```
secure-login> db.users.find({})
[
  {
    _id: ObjectId('69e509ea943e14b34ffe0b25'),
    username: 'Mahek',
    email: 'mahekv4306@gmail.com',
    password: '$2b$10$YuJ3II9/dZnV7.efaGiOf.Hgt4y/i4WKYd5KcCvhyffyttN9ECfT2',
    verificationCode: null,
    verificationCodeExpires: null,
    resetPasswordToken: '2d4273e71bc2659775b6a051bfa37b0792554e96a25e6db0845d8586778f08ef',
    resetPasswordExpires: ISODate('2026-04-19T17:21:14.971Z'),
    isVerified: true,
    createdAt: ISODate('2026-04-19T16:59:22.835Z'),
    updatedAt: ISODate('2026-04-19T17:06:14.974Z'),
    __v: 0
  }
]
secure-login>
```

Figure 11: MongoDB — User document showing the SHA-256 hashed reset token and expiry timestamp stored in the database

Phase 2 — Submitting the New Password

22. **Step 1:** The user follows the reset link. A POST request is sent to /bidsphere/user/resetpwd with the raw token and email as query parameters, and newPassword and confirmNewPassword in the request body.
23. **Step 2:** The server re-hashes the incoming raw token using SHA-256 and compares the result against the stored hash. A match confirms token authenticity.
24. **Step 3:** The expiry field is checked. If the current timestamp exceeds resetPasswordExpires, the reset is rejected with an “Invalid or expired token” error.
25. **Step 4:** bcrypt.compare() ensures the new password is not identical to the current one, preventing trivial bypass.
26. **Step 5:** The new password is hashed with bcryptjs and saved. The resetPasswordToken and resetPasswordExpires fields are cleared to null, invalidating the token permanently.

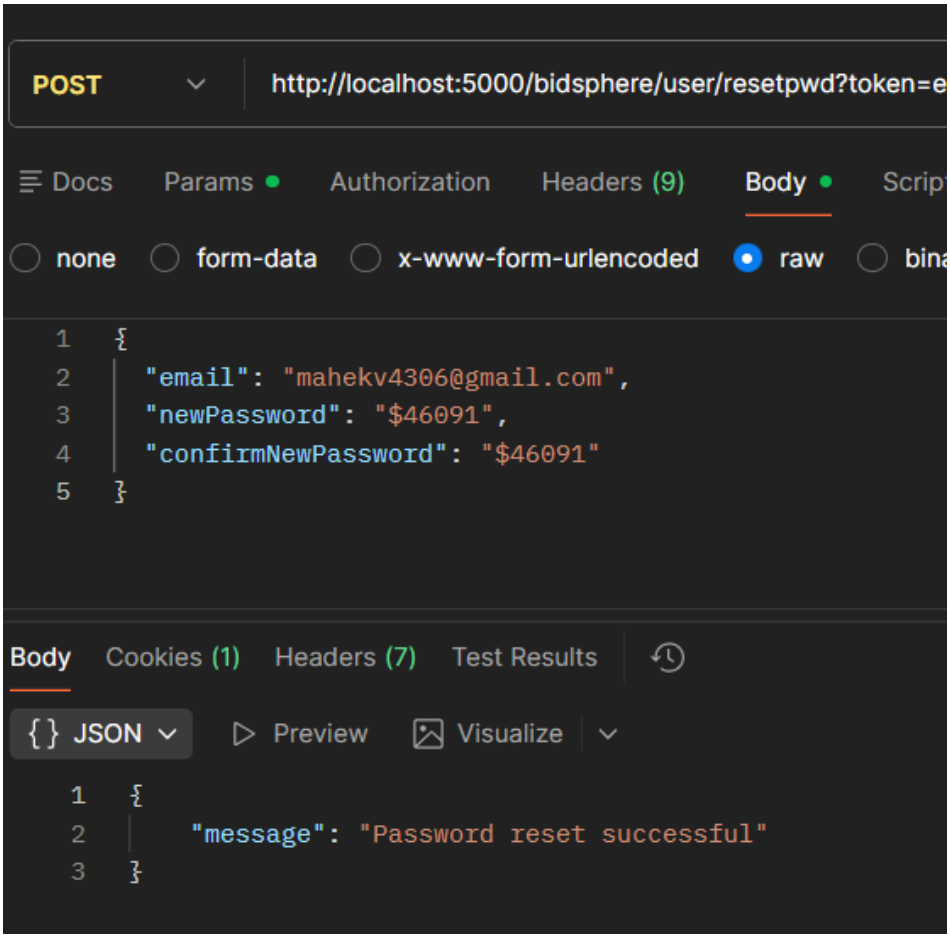

Figure 12: Postman — Successful password reset after submitting the new credentials

```
secure-login> db.users.find({})
[
  {
    _id: ObjectId('69e509ea943e14b34ffe0b25'),
    username: 'Mahek',
    email: 'mahekv4306@gmail.com',
    password: '$2b$10$D9Cf1Zqo4fmd5f8M75zD5uQ/fb4XtzPXUodxai11YQK9koETQli0S',
    verificationCode: null,
    verificationCodeExpires: null,
    resetPasswordToken: null,
    resetPasswordExpires: null,
    isVerified: true,
    createdAt: ISODate('2026-04-19T16:59:22.835Z'),
    updatedAt: ISODate('2026-04-19T17:07:45.841Z'),
    __v: 0
  }
]
secure-login>
```

Figure 13: MongoDB — Token fields cleared to null and updated password hash stored after reset

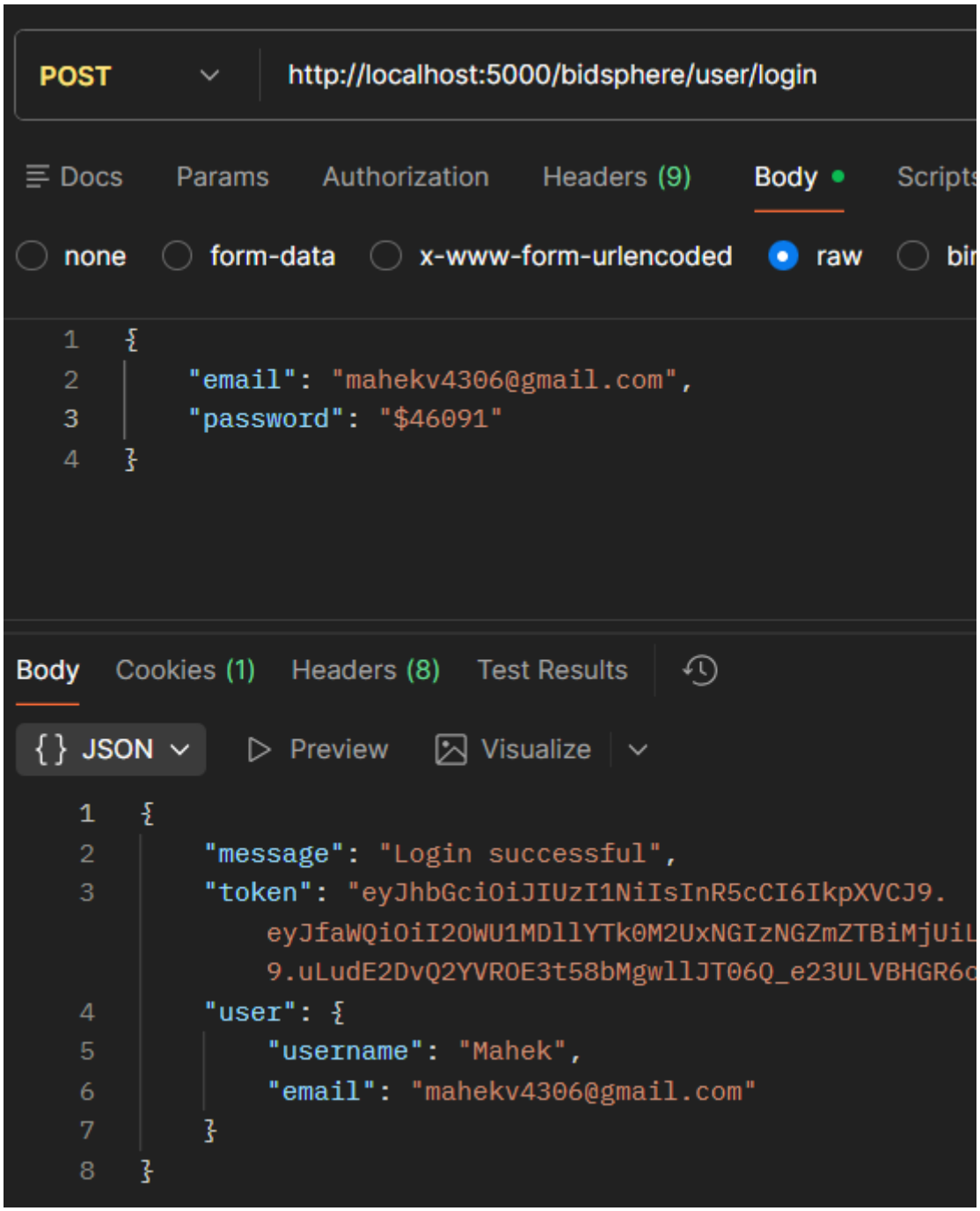

Figure 14: Postman — Login with the newly reset password confirms the update was applied successfully

3. Security Mechanisms

This section details the individual security mechanisms applied throughout the authentication system. Each mechanism addresses a specific attack vector and contributes to the overall defence-in-depth posture of the application.

3.1 Password Hashing with bcryptjs

bcryptjs implements the bcrypt hashing algorithm, which is specifically designed for password storage. Unlike general-purpose hashing algorithms such as MD5 or SHA-256, bcrypt incorporates an adaptive cost factor and an automatic salt, making brute-force and rainbow table attacks computationally infeasible.

How bcrypt Protects Passwords
• Salt rounds: A cost factor of 10 is applied, meaning 2^{10} iterations are performed per hash. This makes offline cracking slow.
• Unique salt: bcrypt generates a unique random salt for each hash, ensuring that two users with the same password produce different hash values.
• One-way function: The hash cannot be reversed. Verification is performed by re-hashing the input and comparing — the original password is never reconstructed.
• Secure comparison: bcrypt.compare() uses a constant-time algorithm, preventing timing-based side-channel attacks.

The bcrypt hash visible in the MongoDB screenshots follows the format $2b$10$... where $2b identifies the algorithm version, $10 is the cost factor, and the remaining characters encode the salt and hash together. Stored passwords are always in this format — never in plaintext.

3.2 Token Generation and SHA-256 Hashing (crypto)

The password reset system uses Node.js's built-in crypto module to manage token security. The approach separates what is sent from what is stored, providing protection even in the event of a database compromise.

Token Security Model
• Generation: crypto.randomBytes(32) produces 32 bytes of cryptographically secure randomness, hex-encoded as a 64-character string.
• Hashing: The raw token is immediately hashed via crypto.createHash('sha256'). Only this hash is written to the database.
• Transmission: The unhashed token is embedded in the reset URL sent by email. It never touches the database.

• Verification: When the user returns with the token, the server re-applies SHA-256 and compares the result to the stored hash.
• Benefit: If the database is compromised, an attacker cannot reconstruct the valid reset URL because they only have the hash.

3.3 Token Expiry and Validation

Time-limited tokens are a critical security control in any reset or one-time-use flow. The system enforces a strict 15-minute expiry window for password reset tokens.

- **Expiry creation:** The resetPasswordExpires timestamp is set to Date.now() + 15 minutes at the moment of token generation.
- **Expiry enforcement:** At the time of reset, the server checks whether resetPasswordExpires.getTime() < Date.now(). If expired, the request is rejected with a clear error.
- **Single-use guarantee:** Once a password is reset, both token fields are set to null. A token cannot be used a second time.
- **Token rotation:** If a new reset is requested before the window expires, the previous token is overwritten in the database, invalidating the old link.

3.4 JWT-Based Session Management

After a successful login, a JSON Web Token is issued. This token enables stateless authentication across all subsequent API requests without requiring server-side session storage.

JWT Configuration and Cookie Security
• Payload: The JWT encodes the user's _id and email. Sensitive fields such as the password hash are never included.
• Secret: The token is signed using a server-side secret (JWT_SECRET) stored in environment variables, making forgery detectable.
• Expiry: Tokens expire after 7 days, limiting the window of exposure if a token is intercepted.
• HttpOnly cookie: The token is set as an HttpOnly cookie, preventing JavaScript from reading it and mitigating XSS attacks.
• Secure + SameSite=None: These flags ensure the cookie is only transmitted over HTTPS and is protected from CSRF exploitation.
• DB exclusion: When resolving a token, the user object returned excludes password, resetPasswordToken, and resetPasswordExpires fields using Mongoose’s .select() method.

3.5 Information Leakage Prevention

A critical but often overlooked security practice is ensuring that error responses do not reveal whether a specific account exists. BidSphere applies this consistently across the login flow.

- **Generic error messages:** For both a non-existent email and an incorrect password, the system returns the identical response: “Invalid email or password” with HTTP 401. This prevents account enumeration attacks.
- **Verification sequence:** Email verification status is checked only after password validation. The sequence prevents an attacker from determining whether an unverified email is registered.

5. Conclusion

The authentication module demonstrates how a backend application can be engineered with security as a foundational concern rather than an afterthought. By combining bcryptjs for password hashing, the crypto module for token generation and SHA-256 hashing, JWT for stateless session management, and Nodemailer for transactional email delivery, the system achieves a layered security model that addresses a broad range of common attack vectors.

No plaintext password is ever stored or transmitted. No raw reset token is ever persisted in the database. Error messages are deliberately generic to prevent account enumeration. Session tokens are signed, scoped, and protected by cookie security flags. Reset tokens are time-limited and single-use. Each of these design choices is intentional and maps to a concrete security requirement.

The full workflow — from registration through verification, login, and password recovery — was validated through Postman testing and MongoDB shell inspection. All tests confirmed that the system responds correctly in both success and failure scenarios, and that the database state at each stage reflects the expected security properties.

— End of Report —